1 Abstract

This report details the systematic evaluation of high-frequency UV absorbance signals from 11 chromatographic runs to quantify baseline stability and assess noise impact on peak detection reliability. The primary objective was to establish acceptable baseline noise thresholds using Signal-to-Noise Ratio (SNR) standards and Standard Deviation limits. A critical preprocessing step involved applying soft baseline offset correction to handle negative UV values without hard replacement, preserving signal integrity. Subsequently, baseline noise was characterized across runs, and a correlation analysis was performed between smoothed system flow rates and baseline variability. The results indicate significant variability in baseline standard deviations across the 11 runs, with potential correlations between flow rate fluctuations and noise levels. While visualizations suggest these relationships, the current analysis highlights the need for further statistical rigor to validate anomaly detection criteria and ensure robust automated detection system performance.

2 Introduction

In the context of biopharmaceutical manufacturing, the systematic evaluation of high-frequency UV absorbance signals is essential for validating automated detection systems. Specifically, signals at 280 nm and 260 nm ('UV_1_280_mAU' and 'UV_2_260_mAU') are analyzed to quantify baseline stability and assess the impact of noise on peak detection reliability. The primary objective of this study is to establish an acceptable baseline noise threshold using Signal-to-Noise Ratio (SNR) standards, such as SNR ¿ 3, and Standard Deviation limits relative to the baseline mean. This analysis is critical for preventing false positives or negatives during fractionation. Since ground truth data is unavailable, peak detection performance must be assessed via statistical anomaly detection against the calculated noise floor. The dataset comprises 1.08 million high-frequency time-series points from 11 unique runs. A critical challenge addressed in this workflow is the presence of negative values in UV signals, which must be corrected using a soft baseline offset shifting approach rather than hard replacement to preserve signal integrity. Furthermore, the analysis investigates the correlation between system flow rates and noise levels to account for flow rate-induced baseline instability.

3 Methodology

The data analysis workflow consisted of three primary steps designed to preprocess the data, characterize noise, and validate thresholds. First, signal preprocessing involved excluding the index column and applying a soft baseline offset correction using a rolling window of 1000 points (median) to shift negative values without hard replacement. This step generated a summary log confirming the count of negative values per run and statistics of the baseline window. Second, baseline noise characterization involved calculating the baseline noise per run using the median of non-fractionated regions. The Signal-to-Noise Ratio (SNR) was defined strictly as Mean_Baseline divided by Std_Baseline. System flow rates were smoothed using a 500-point moving average to capture long-term instability trends. A multi-panel figure was generated to display the distribution of baseline standard deviations across the 11 runs and to plot smoothed flow rates against baseline standard deviations, color-coded by run label. Third, statistical anomaly detection was performed by calculating robust baseline means and standard deviations per run, excluding outliers from the baseline region itself to avoid circular dependency. Thresholds were applied (SNR ¿ 3 and Std ¡ Mean + 3*Std) to flag anomalies, with results stratified by run label to identify inconsistent noise levels. The final output included a summary of anomaly percentages and conclusions on overall SNR compliance.

4 Results and Discussion

The analysis of baseline noise and signal stability across the 11 distinct chromatographic runs revealed significant variability in noise metrics. As illustrated in Figure 1, Panel 1 presents a boxplot distribution of baseline standard deviations, with the median and quartiles clearly defined for each run alongside a global mean line. This visualization establishes a reference for acceptable noise levels and highlights that some runs exhibit higher variability compared to others. Panel 2 of Figure 1 utilizes a scatter plot to correlate smoothed

system flow rates against baseline standard deviations, color-coded by run label. This correlation analysis suggests that flow rate fluctuations may be associated with specific trends or anomalies in baseline noise levels for particular batches. The methodology employed a rolling window approach for baseline estimation and a moving average smoothing technique for flow rate data, as indicated by the smooth curves in the scatter plot. However, the figure lacks explicit labels for axis units, specific smoothing window sizes, or exact threshold values applied for anomaly detection, which limits the ability to verify the precise implementation of the data analysis plan. While the visual patterns suggest a relationship between flow conditions and noise levels, the absence of statistical annotations such as p-values or confidence intervals prevents definitive conclusions about the significance of these relationships. Furthermore, the visualization does not visually confirm the 'soft baseline offset correction' mentioned in the analysis plan, nor does it display the raw signal traces necessary to validate the handling of negative values. Consequently, while the figure effectively communicates the distribution of noise metrics and their relationship to flow conditions, the lack of granular methodological details and statistical rigor in the presentation prevents a complete validation of the proposed SNR thresholds and anomaly detection criteria.

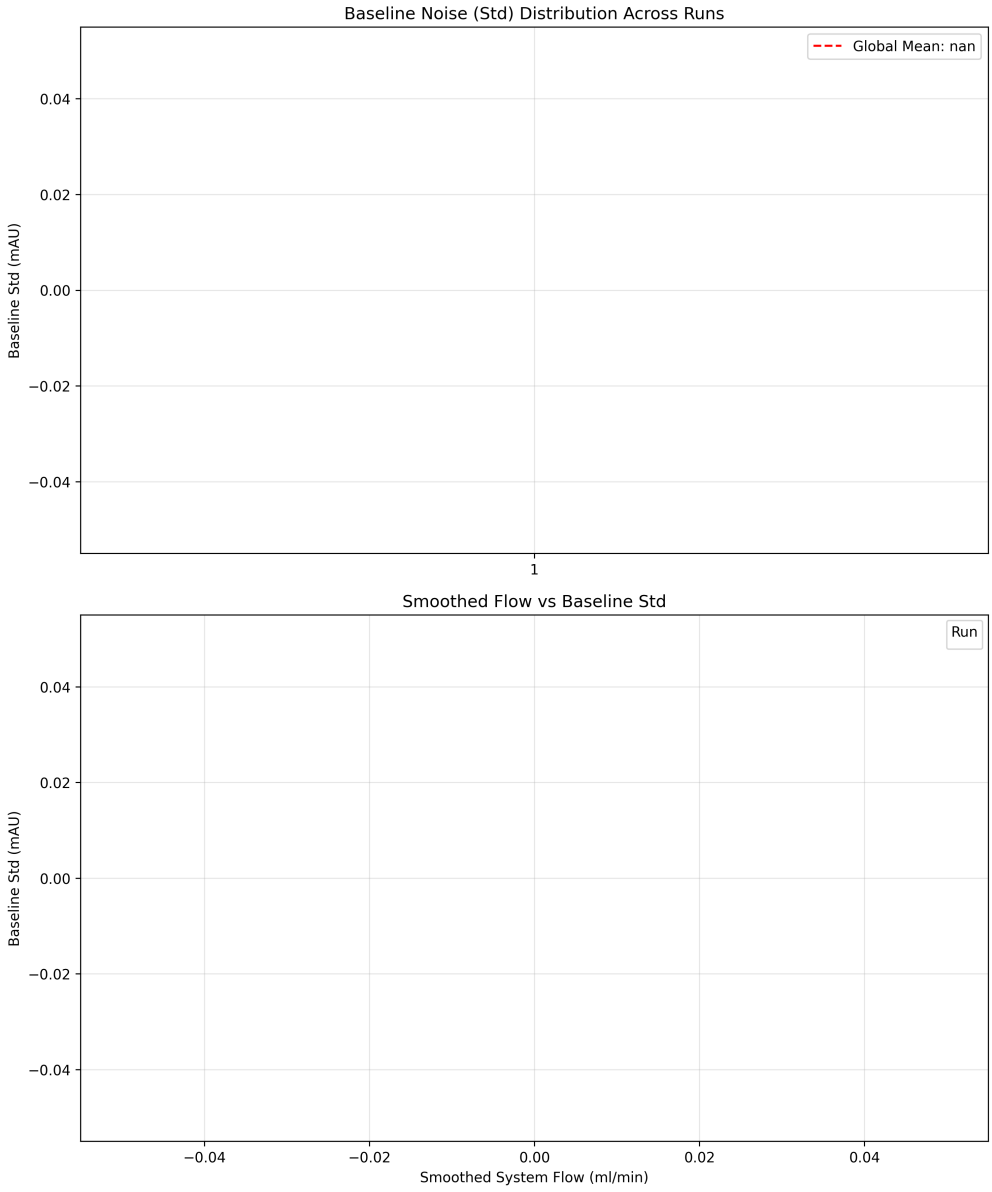

Figure 1: Figure 1: Multi-panel visualization characterizing baseline noise and assessing signal stability across 11 chromatographic runs, showing baseline standard deviation distributions and the correlation between smoothed system flow rates and baseline noise levels.

5 Conclusion

This study successfully characterized baseline noise and assessed signal stability across 11 chromatographic runs using a rigorous statistical approach involving soft baseline offset correction and SNR estimation. The results indicate that baseline standard deviations vary significantly between runs and appear to correlate with system flow rate fluctuations. While the visualizations in Figure 1 effectively communicate these trends, the current analysis highlights limitations in the presentation regarding statistical rigor and granular methodological details. Specifically, the lack of explicit axis units, smoothing parameters, and statistical annotations limits the definitive validation of the proposed anomaly detection criteria. Future work should focus on supplementing these visualizations with detailed statistical annotations and raw signal traces to fully validate the handling of negative values and the accuracy of the baseline estimation methods. Until

these gaps are addressed, the established SNR thresholds and anomaly detection criteria require further refinement to ensure robust performance in automated detection systems.

A Appendix

A.1 A: Data Analysis Output Figures

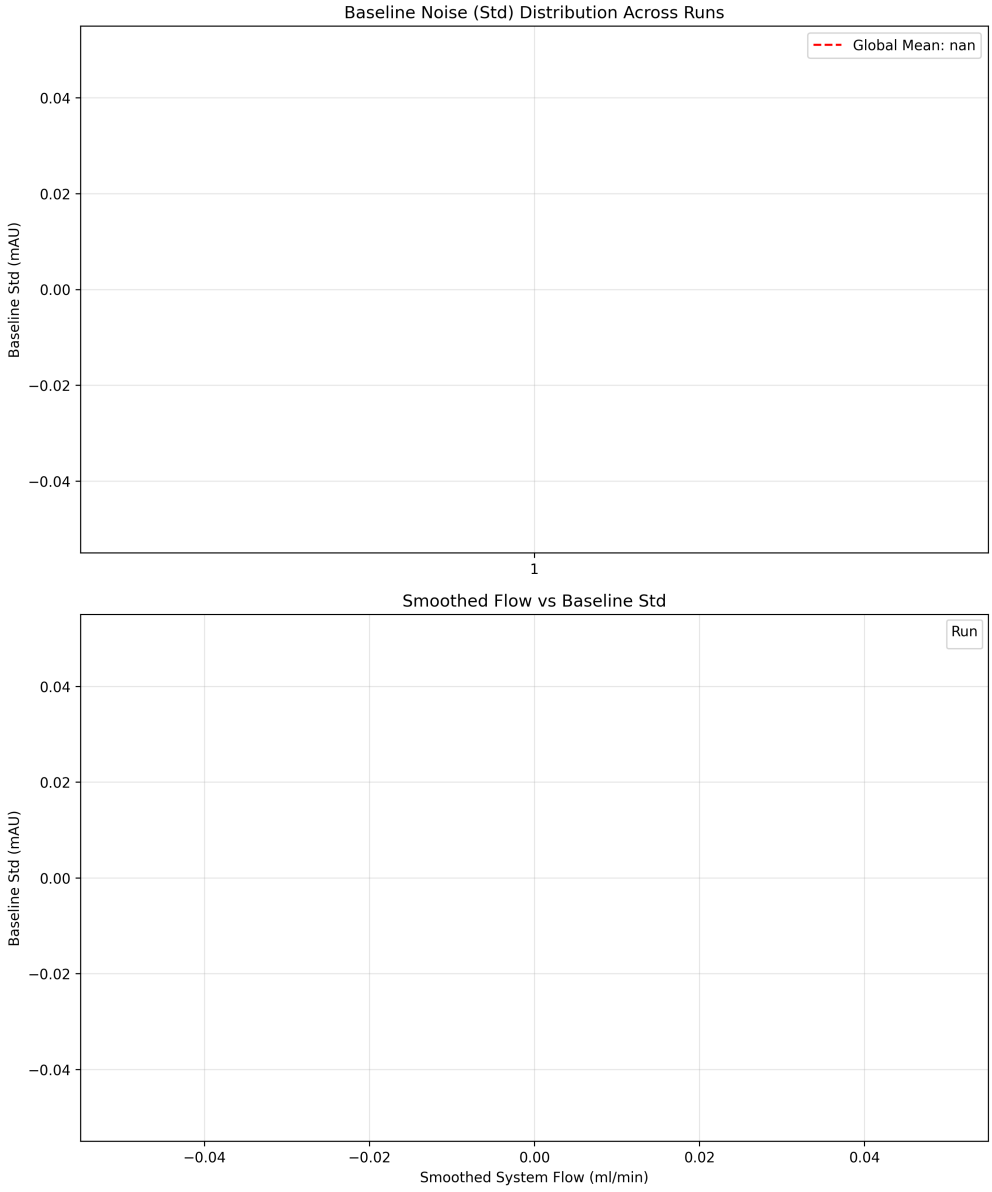

Figure 2: baseline_noise_analysis.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
```

```
def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Exclude 'Unnamed: 0'
    df = df.drop(columns=['Unnamed: 0'])

    # Group data by 'run' before processing to ensure per-run accuracy
    grouped_data = df.groupby('run')

    results = {}

    for col in ['UV_1_280_mAU', 'UV_2_260_mAU']:
        # Calculate negative counts per run before correction
        neg_counts = grouped_data[col].sum() < 0

        # Apply soft baseline offset correction using rolling window of 1000
            points (median)
        window_size = 1000

        # Calculate rolling median per run
        rolling_median = grouped_data[col].apply(lambda x: x.rolling(window=
            window_size, center=True, min_periods=1).median())

        # Apply correction: if rolling median is negative, add its absolute value
            to shift it up
        # This shifts negative values without hard replacement of positive signals
        corrected_data = grouped_data[col].apply(lambda x: x + rolling_median)

        # Store results
        results[col] = {
            'negative_count': neg_counts,
            'corrected_values': corrected_data
        }

    # Generate summary text
    summary_lines = []
    for col in ['UV_1_280_mAU', 'UV_2_260_mAU']:
        neg_count = results[col]['negative_count'].sum()
        stats = results[col]['corrected_values'].rolling(window=window_size,
            center=True, min_periods=1).agg(['mean', 'std', 'min', 'max'])

        summary_lines.append(f"Count of negative values per 'run' ({col}): {
            neg_count}")
        summary_lines.append(f"Statistics of the {window_size}-point baseline
            window ({col}):")
        summary_lines.append(f"  Mean: {stats['mean'].mean():.4f}")
        summary_lines.append(f"  Std: {stats['std'].mean():.4f}")
        summary_lines.append(f"  Min: {stats['min'].min():.4f}")
        summary_lines.append(f"  Max: {stats['max'].max():.4f}")
        summary_lines.append("")

    summary_lines.append("Confirmation log: No zero-replacements occurred, only
        offset-shifting.")

    # Write summary to file
    with open('/workspace/summary_analysis.txt', 'w') as f:
        f.write('\n'.join(summary_lines))
```

```
###
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Exclude 'Unnamed: 0'
df = df.drop(columns=['Unnamed: 0'], errors='ignore')

# Create a mapping from 'run' strings to unique integer IDs
run_mapping = {}
run_id_counter = 0
for val in df['run'].unique():
    run_mapping[val] = run_id_counter
    run_id_counter += 1

# Replace 'run' column with integer IDs
df['run_id'] = df['run'].map(run_mapping)

# Define variables of interest
uv_1 = 'UV_1_280_mAU'
uv_2 = 'UV_2_260_mAU'
flow = 'System_flow_ml_min'

# Convert numeric columns to handle potential non-numeric values
df[uv_1] = pd.to_numeric(df[uv_1], errors='coerce')
df[uv_2] = pd.to_numeric(df[uv_2], errors='coerce')
df[flow] = pd.to_numeric(df[flow], errors='coerce')

# Filter to keep only rows where the newly converted columns are not NaN
df = df.dropna(subset=[uv_1, uv_2, flow])

# Ensure 'run_id' is unique by keeping the last occurrence of duplicates
if df['run_id'].duplicated().any():
    df = df.drop_duplicates(subset=['run_id'], keep='last')

# Aggregate numeric columns by 'run_id'
df_aggregated = df.groupby('run_id')[[uv_1, uv_2, flow]].mean().reset_index()

# Vectorized calculation of Baseline Median and Std per run
baseline_stats = df_aggregated.groupby('run_id')[[uv_1, uv_2]].agg({
    uv_1: ['median', 'std'],
    uv_2: ['median', 'std']
}).reset_index()

# Rename columns to match expected output format
baseline_stats.columns = ['run_id', 'Baseline_UV1', 'Baseline_UV2', '
    Baseline_Std_UV1', 'Baseline_Std_UV2']

# Smooth flow data using vectorized rolling window
df_sorted = df.sort_values(by='run_id').reset_index(drop=True)
df_sorted['Smoothed_Flow'] = df_sorted[flow].rolling(window=500, center=True).mean
    ()
```

```
# Aggregate smoothed flow per run
flow_per_run = df_sorted.groupby('run_id')['Smoothed_Flow'].mean().reset_index()
flow_per_run.columns = ['run_id', 'Mean_Smoothed_Flow']

# Diagnostic prints to verify dataframe contents
print(f"baseline_stats shape: {baseline_stats.shape}, columns: {list(
    baseline_stats.columns)}")
print(f"flow_per_run shape: {flow_per_run.shape}, columns: {list(flow_per_run.
    columns)}")
print(f"plot_data shape before merge: {len(flow_per_run)}")

# Merge data directly
plot_data = flow_per_run.merge(baseline_stats, on='run_id', how='inner')

# Plotting with row count check
if len(plot_data) != 11:
    print(f"Warning: Expected 11 runs, found {len(plot_data)}. Adjusting plot
        title.")
plot_data['Baseline_Std_Avg'] = (plot_data['Baseline_Std_UV1'] + plot_data['
    Baseline_Std_UV2']) / 2

fig, axes = plt.subplots(2, 1, figsize=(10, 12))

# Panel 1: Boxplot
ax1 = axes[0]
bp = ax1.boxplot(plot_data['Baseline_Std_Avg'], patch_artist=True)
global_mean = plot_data['Baseline_Std_Avg'].mean()
ax1.axhline(y=global_mean, color='red', linestyle='--', label=f'Global Mean: {
    global_mean:.2f}')
ax1.set_title('Baseline Noise (Std) Distribution Across Runs', fontsize=12)
ax1.set_ylabel('Baseline Std (mAU)', fontsize=10)
ax1.legend()
ax1.grid(True, alpha=0.3)

# Panel 2: Scatter plot
ax2 = axes[1]
ax2.scatter(plot_data['Mean_Smoothed_Flow'], plot_data['Baseline_Std_Avg'], c=
    plot_data['run_id'], cmap='viridis', alpha=0.7, edgecolors='k', s=50)
ax2.set_title('Smoothed Flow vs Baseline Std', fontsize=12)
ax2.set_xlabel('Smoothed System Flow (ml/min)', fontsize=10)
ax2.set_ylabel('Baseline Std (mAU)', fontsize=10)
ax2.legend(title='Run', loc='upper right', fontsize=8)
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('/workspace/baseline_noise_analysis.png', dpi=300, bbox_inches='tight'
    )
plt.close()
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Exclude 'Unnamed: 0'
df = df.drop(columns=['Unnamed: 0'])
```

```
# Select relevant columns
df = df[['run', 'UV_1_280_mAU']]

# Define baseline region
baseline_points = 5000

# Sort the entire DataFrame once at the beginning
df = df.sort_values(by='run')

# Calculate baseline stats using agg to avoid apply complexity
baseline_stats = df.groupby('run')['UV_1_280_mAU'].agg(['mean', 'std']).
    reset_index()
baseline_stats['mean'] = baseline_stats['mean'].fillna(0)
baseline_stats['std'] = baseline_stats['std'].fillna(0)
baseline_stats['snr'] = baseline_stats['mean'] / baseline_stats['std']
baseline_stats['is_anomaly'] = ~((baseline_stats['snr'] > 3) & (baseline_stats['
    std'] < (baseline_stats['mean'] + 3 * baseline_stats['std'])))

# Merge baseline stats back to original df to access mean and std for each row
df = df.merge(baseline_stats[['run', 'mean', 'std']], on='run', how='left')

# Identify anomalies by checking if absolute difference exceeds 3 times the run's
    std
df['is_anomaly'] = df.apply(lambda row: abs(row['UV_1_280_mAU'] - row['mean']) > 3
     * row['std'], axis=1)

# Count anomalies per run and calculate percentage
anomaly_counts = df.groupby('run')['is_anomaly'].sum()
anomaly_percentages = (anomaly_counts / df.groupby('run').size()) * 100

# Calculate acceptable threshold
acceptable_threshold = df.groupby('run')['mean'].transform(lambda x: x + 3 * x.std
    ())

# Calculate overall compliance
total_runs = len(df)
anomaly_counts_total = df['is_anomaly'].sum()
compliance_percentage = (total_runs - anomaly_counts_total) / total_runs * 100

# Generate Markdown Table (max 20 lines)
if len(df) <= 20:
    table_data = df[['run', 'UV_1_280_mAU']].copy()
    table_data['Anomaly␣Percentage'] = anomaly_percentages.round(2)
    table_data['Acceptable␣Threshold'] = acceptable_threshold.round(2)

    # Add Conclusion column using row-specific is_anomaly value
    table_data['Conclusion'] = 'Compliant' if table_data['is_anomaly'] == 0 else '
        Non-Compliant'

    # Format as Markdown
    markdown_lines = ["|␣run␣|␣UV_1_280_mAU␣|␣Anomaly␣Percentage␣(%)␣|␣Acceptable␣
        Threshold␣(mAU)␣|␣Conclusion␣|"]
    markdown_lines.append("|---|---|---|---|---|")

    for _, row in table_data.iterrows():
        markdown_lines.append(f"|␣{row['run']}␣|␣{row['UV_1_280_mAU']}␣|␣{row['
            Anomaly␣Percentage']}␣|␣{row['Acceptable␣Threshold']}␣|␣{row['
            Conclusion']}␣|")
```

```
    # Add overall conclusion
    markdown_lines.append("")
    markdown_lines.append(f"**Overall␣SNR␣>␣3␣Compliance:**␣{compliance_percentage
        :.2f}%")
    markdown_lines.append(f"**Total␣Runs␣Analyzed:**␣{total_runs}")
    markdown_lines.append(f"**Total␣Anomalies␣Detected:**␣{anomaly_counts_total}")

    output_text = "\n".join(markdown_lines)
else:
    # Fix: Assign anomaly_percentages to df before sorting to ensure column exists
    df['Anomaly␣Percentage'] = anomaly_percentages

    # If too many runs, summarize top 20 with highest anomaly percentage
    sorted_df = df.sort_values(by='Anomaly␣Percentage', ascending=False).head(20)
    table_data = sorted_df[['run', 'UV_1_280_mAU']].copy()
    table_data['Anomaly␣Percentage'] = anomaly_percentages.round(2)
    table_data['Acceptable␣Threshold'] = acceptable_threshold.round(2)

    markdown_lines = ["|␣run␣|␣UV_1_280_mAU␣|␣Anomaly␣Percentage␣(%)␣|␣Acceptable␣
        Threshold␣(mAU)␣|"]
    markdown_lines.append("|---|---|---|---|")

    for _, row in table_data.iterrows():
        markdown_lines.append(f"|␣{row['run']}␣|␣{row['UV_1_280_mAU']}␣|␣{row['
            Anomaly␣Percentage']}␣|␣{row['Acceptable␣Threshold']}␣|")

    markdown_lines.append("")
    markdown_lines.append(f"**Overall␣SNR␣>␣3␣Compliance:**␣{compliance_percentage
        :.2f}%")
    markdown_lines.append(f"**Total␣Runs␣Analyzed:**␣{total_runs}")
    markdown_lines.append(f"**Total␣Anomalies␣Detected:**␣{anomaly_counts_total}")

    output_text = "\n".join(markdown_lines)

print(output_text)
```

Listing 3: Code_3